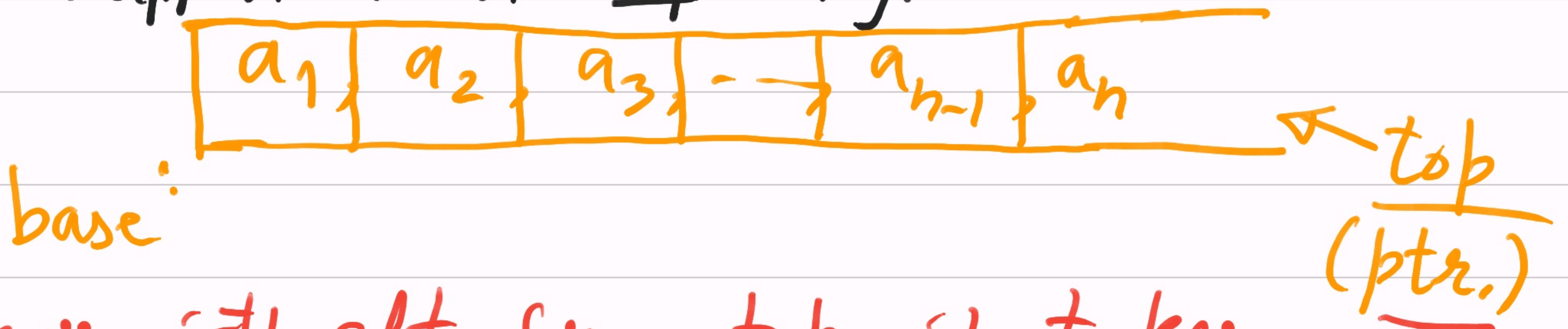


WAP: Input: expression (say in an array).
Output: value.

→ An $O(n)$ algo. for this inspires the ds stack.

Math. modeling: Stack is a list where all ops. happen at the top only.



ie. to access i -th elt. from top, it takes $O(i)$ time; go through $a_n, a_{n-1}, \dots, a_{n-i+1}$.

- Query ops.:
 - $\text{isEmpty}(S)$ // is Stack $S \stackrel{?}{=} \emptyset$?
 - $\text{top}(S)$ // returns the top elt. a_n .
- Update ops.:
 - $\text{createEmptyStack}(S)$
 - $\text{push}(x, S)$ // push x at the top of the stack S ; change the top ptr.
 - $\text{pop}(S)$ // delete & output the top elt. of S ; change the top ptr.

e.g. $\text{push}(x, [a_1, a_2]) = [a_1, a_2, x]$.
 $\text{pop}([a_1, a_2, a_3]) = a_3 \ \& \ [a_1, a_2]$.

- Why does 1-side-access help in ^{node} S^{new} expression-evaluation?

- ▷ Stack can be implemented as : array, or as a doubly-link-list (preferable).
- ▷ Ops. take only $O(1)$ time.

- Let's see the idea for expr. evaluation:
 - eg. $x = 8 + 3 * 5^2 - 9$. (is unambiguous)

Precedence: $\overset{\text{operand} \uparrow}{\boxed{\wedge}} > \boxed{\div} > \boxed{*} > \boxed{+} = \boxed{-}$

- Stack provides a very clean pseudocode.

8	+	3	*	5	^	2	-	9	
---	---	---	---	---	---	---	---	---	--

 ← top.

$o_1 < o_2 < o_3 > o_4$

- Let's see the schematics with 1 stack.

Idea: Evaluate the op. O_i immediately, if O_i is a local maxima wrt precedence.

- e.g. $O_3 = \wedge$ is a local maxima.
inaccessible

$[8 + 3 * 5 \wedge 2]$
 $O_1 < O_2 < O_3 > O_4$

[pop -, 2, ^, 5 ; push 25]

$[8 + 3 * 25 = 75]$
 $O_1 < O_2 > O_3$

[pop -, 25, *, 3 ; push 75]

$[8 + 75 = 83]$
 $O_1 = O_2$

$[8 + 66 = 74]$ (Stop.)

▷ [Invariants] • The elements in the "base" do not have local maxima, i.e. non-decreasing.

(else, we would've popped it before.)

• When we push the calculated value, we don't change the value of the expression x .

(use the unambiguity of x .)

- bad. eg. $y = 5^2 2^3$ is ambiguous as $y = 25^3$ or 5^8 .

→ use "left associativity": $y = (5^2)^3$.

$$y' = 5^{(2^3)} \neq y.$$

- To make the code clearer, use 2 stacks:

N-stack (operands):

9
2
5
3
8

O-stack :
(operators)

-
^
*
+

Exercise: • Write the pseudo code.

- Handle parentheses (&) .

[Idea: '(' means an artificial-base of the stack.
') ' " that the substack value has to
be calculated first.]

- Stack is how^{the} Operating System (OS) implements
your recursive C code.

call1 | call2 | call3 | ---

- Behavior of Stack: LIFO (Last In, First Out)
- Let's do now : FIFO (First In, " ")

Queue

- Queue has a front & a rear.

- Math. modeling.

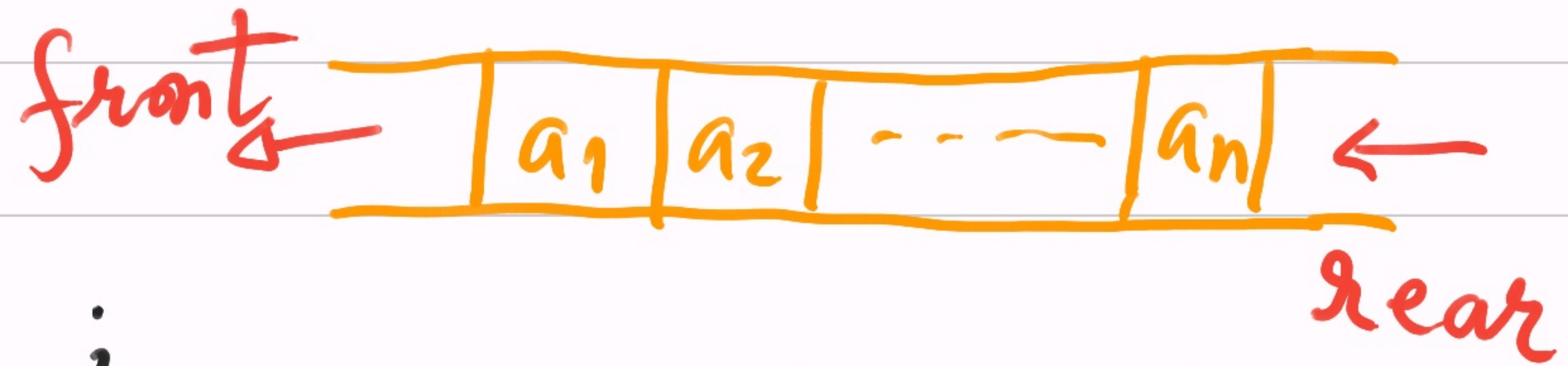
Query ops. : • isEmptY(Q) ;

• front(Q)

// return the value at the front

Update ops. : • createEmptY Queue(Q) ;

• enqueue(x, Q) ; // insert x at the rear

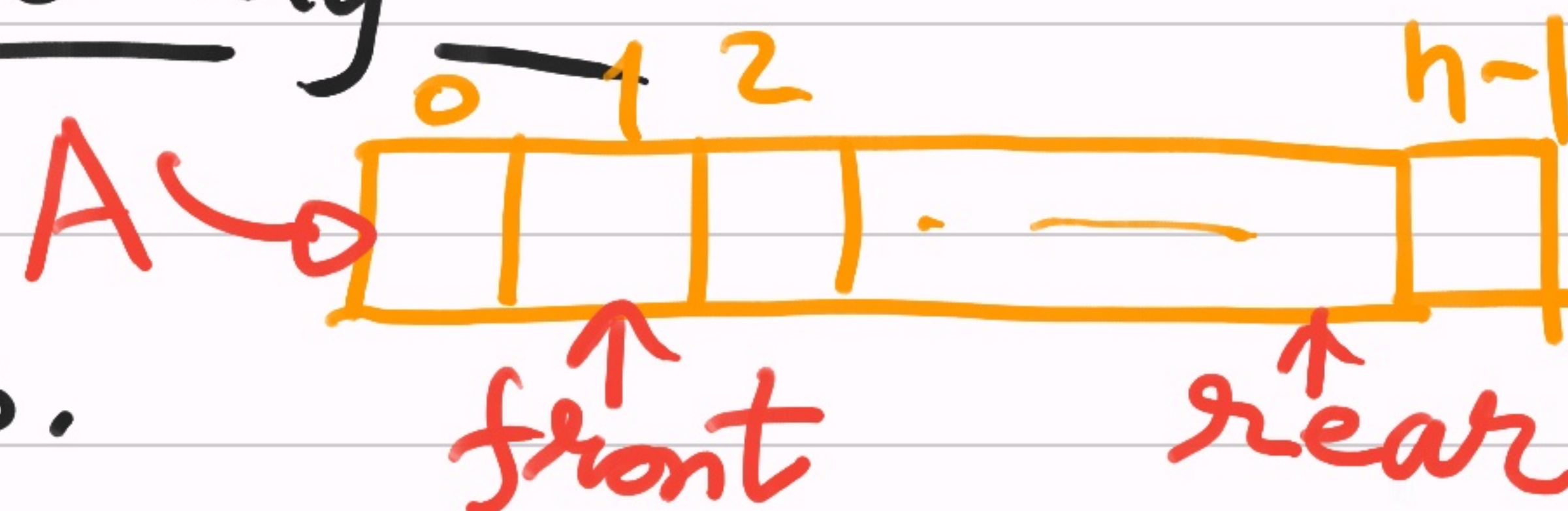


- $\text{dequeue}(Q)$; // delete & return the front value
- e.g. $\text{enqueue}(x, (a_1, \dots, a_n)) = (a_1, \dots, a_n, x)$
"a_{n+1}"
- $\text{dequeue}((a_1, \dots, a_n)) = a_1 \ \& \ (a_2, \dots, a_n)$.
- ↑ front ↑ rear

▷ Accessing a_i takes $O(i)$ time.

Implement Q using an array

- Assume $|Q| \leq n$ at all times.



- Qn: What to do when rear goes to n from $n-1$?

Idea: Loop around! $\text{rear} \leftarrow (\text{rear} + 1) \bmod n$; ^{enqueue}
 $\text{front} \leftarrow (\text{front} + 1) \bmod n$; ^{dequeue}

Exercise: 1) Implement a stack using a queue;
in $O(n)$ -time.

(2) Queue using a stack? No.

[Cheat code: Use recursion on the stack code.]

(3) Implement queue using two stacks;
in $O(n)$ -time.

- Let's use Q in a real problem.

Shortest path in a maze (grid with obstacles)

WAP: Input: • Grid $G[0 \dots n-1][0 \dots n-1]$ with $\forall i, j$,
 $G[i][j] = 0$, if $(i, j) \in \text{obstacle}$
 $= 1$, else

• Cells s & t .

Output: • Find the shortest route from s to t .
• In one step, move to a neighboring cell
(one out its four).

Idea: [Brute-force] Move & backtrack!

exponential-time

• [Iterative] Create a ripple from s & let the ripple move forward incrementally.

• The ripple gives layers $L_0, L_1, L_2, \dots$ (till you reach t)

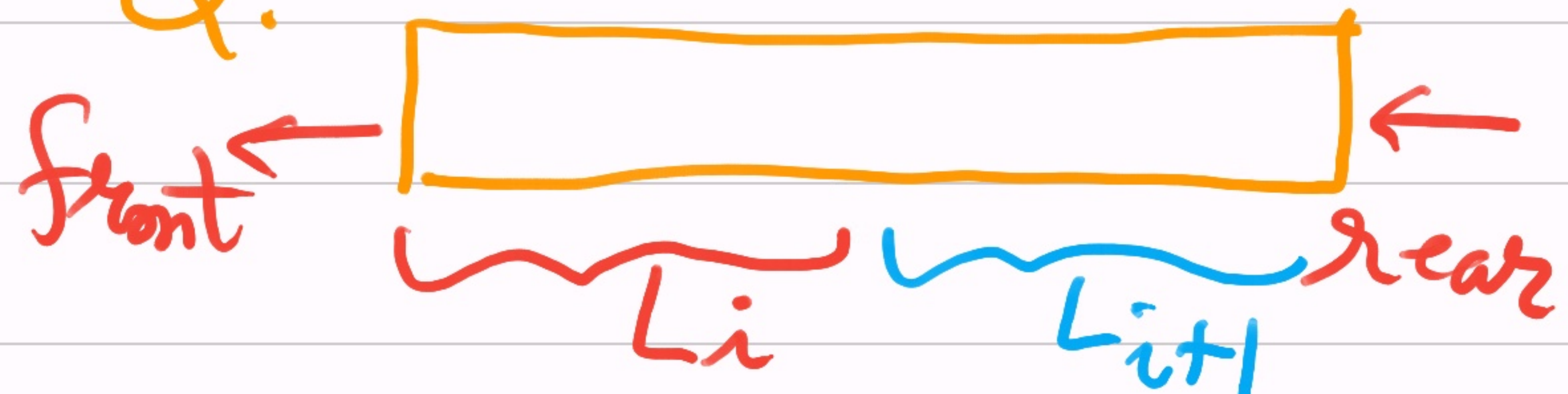
$d(\cdot) = 0, 1, 2, 3, 4, 5, 6$



→ Use a queue to store this & go from L_i to L_{i+1} :

Q:

• Dequeue $c \in L_i$ & enqueue $b \in \text{nbrd}(c)$ in Q.



Pseudocode: gridDistance (G, s, t) {

set $d(c) = \infty, \forall c$;

create Empty Queue (Q);

$d(s) = 0$; enqueue (s, Q);

while (!isEmptyQueue(Q)) {

$c \leftarrow$ dequeue(Q);

for ($b \in \text{nbhd}(c) \setminus \text{obstacle}$)

if ($d(b) = \infty$) {

$d(b) \leftarrow d(c) + 1$;

enqueue(b, Q); }

return $d(t)$; }

▷ Prove the invariant:
 $\forall$ visited cell c ,
 $d(c)$ = shortest distance
from s in G .

Pf: • induction on distance. Δ
• c is enqueued only once.

▷ Time taken $\approx$ the area
of the ripple $s \sim t$.